HydroGrid

Smart Water & Electricity Intelligence Platform

VIVA PREPARATION GUIDE

Full Stack Developer Project • April 2026

1. PROJECT OVERVIEW

HydroGrid is a production-level, full-stack SaaS web application for tracking, analyzing, and optimizing water and electricity consumption using AI-powered insights, predictive analytics, and real-time monitoring. It targets Indian households and utilities with state-wise tariff calculations.

- JWT Authentication with role-based access (User / Admin)
- Real-time dashboard with animated charts (Line, Bar, Pie/Donut)
- AI-powered anomaly detection (Z-score) and 30-day forecasting (Exponential Smoothing)
- Threshold-based color-coded alert system (green / yellow / red)
- State-wise India tariff calculator for electricity & water bills
- WebSocket live metrics feed (10-second push)
- Leaderboard, gamification badges, carbon footprint estimation
- CSV / PDF report export, Admin panel, India GeoJSON map

2. TECH STACK

Layer	Technology	Purpose
Frontend	React 18 + Vite	Fast HMR, component-based UI
Styling	Tailwind CSS 3	Utility-first, no CSS bloat
Charts	Recharts	React-native chart library
Animations	Framer Motion	Smooth, declarative animations
Icons	Lucide React	Lightweight SVG icons
Backend	Node.js + Express.js	Non-blocking I/O, REST API
Database	PostgreSQL (Supabase)	Relational, hosted cloud DB
Auth	JWT + bcryptjs	Stateless, scalable authentication
AI / LLM	Groq API (LLaMA 3-8B)	Fast inference for AI chat
Real-time	WebSocket (ws library)	Live metrics push feed
PDF Export	PDFKit	Server-side PDF generation
Deploy	Docker + docker-compose	Containerized deployment

3. SYSTEM ARCHITECTURE

```
Browser (React 18 + Vite)
     %   HTTP/REST + WebSocket
     %¼
Express.js Server (Node.js, port 5000)
  % % %  Middleware : CORS, express.json, JWT protect
  % % %  Routes     : /api/auth  /api/usage  /api/alerts
  %                /api/reports  /api/admin  /api/ai  /api/ml
  % % %  Controllers !’ Business Logic
  % % %  Utils      : aiEngine.js  groqAI.js  emailService.js
  % % %  WebSocket  : /ws/live  (pushes every 10 s)
     %
     %¼
PostgreSQL on Supabase (pg Pool, SSL)
  % % %  users table
  % % %  usage_data table
  % % %  alerts table
```

The frontend is a React SPA served by Vite (dev) or Nginx (production). All data flows through a RESTful Express API protected by JWT middleware. WebSocket connections deliver live IoT simulations without polling.

4. AUTHENTICATION FLOW

Registration

- Client sends { name, email, password } to POST /api/auth/register
- Server checks duplicate email via SQL: SELECT id FROM users WHERE email=$1
- Password hashed: bcrypt.genSalt(10) then bcrypt.hash(password, salt)
- User inserted into DB, JWT generated (expires in 30 days), returned to client
- Welcome email sent asynchronously via emailService

Login

- Client sends { email, password } to POST /api/auth/login
- Server fetches user by email, runs bcrypt.compare(password, hash)
- On success: generates JWT signed with JWT_SECRET, returns token + user data

Protected Routes

- Frontend Axios interceptor adds Authorization: Bearer <token> to every request
- protect middleware: extracts token !' jwt.verify() !' fetches user from DB !' req.user
- adminOnly middleware: checks req.user.role === "admin", returns 403 if not
- Response interceptor: catches 401 !' clears localStorage !' redirects to /login

Q: Why JWT over sessions?

A: JWT is stateless — no server-side session store needed, scales horizontally. The token itself carries the user identity (id) signed with a secret key.

Q: Security concern with localStorage?

A: Vulnerable to XSS. Mitigation: React auto-escapes JSX output. Better alternative in production is httpOnly cookies to prevent JS access.

5. DATABASE DESIGN

users table

```
id (PK), name, email (UNIQUE), password (bcrypt hash),
role (user|admin), state, settings (JSONB), avatar,
badges (JSONB), created_at
```

usage_data table

```
id (PK), user_id (FK !’ users), type (water|electricity),
value (NUMERIC), unit, timestamp, state/location
```

alerts table

```
id (PK), user_id (FK !’ users), type, severity (green|yellow|red),
message (TEXT), read (BOOLEAN), timestamp
```

Q: Why PostgreSQL instead of MongoDB?

A: Relational data suits this app — users have structured relationships with usage records and alerts. Supabase provides instant hosted PostgreSQL, perfect for rapid deployment.

Q: What is Supabase?

A: An open-source Firebase alternative built on PostgreSQL. Provides hosted DB, auth, storage, and real-time features. We use it with the pg driver (raw parameterized SQL queries).

Q: What is a connection pool?

A: A pool manages multiple persistent DB connections. Instead of opening/closing a connection per query (expensive), the pool reuses connections efficiently. Configured with SSL for Supabase.

6. API ENDPOINTS

Method	Route	Auth	Purpose
POST	/api/auth/register	No	Create account
POST	/api/auth/login	No	Login, get JWT
POST	/api/auth/google	No	Google OAuth
GET	/api/auth/profile	JWT	Get user profile
PUT	/api/auth/profile	JWT	Update profile
POST	/api/usage	JWT	Add usage record
GET	/api/usage	JWT	Get usage history
GET	/api/usage/dashboard	JWT	Dashboard stats
GET	/api/usage/leaderboard	JWT	Efficiency rankings
GET	/api/usage/carbon	JWT	CO2 footprint
GET	/api/usage/tariff-estimate	JWT	Bill cost estimate
GET	/api/usage/map	JWT	State map data
GET	/api/alerts	JWT	Get alerts
PUT	/api/alerts/:id/read	JWT	Mark alert read
DELETE	/api/alerts/:id	JWT	Delete alert
GET	/api/admin/stats	Admin	Platform stats
GET	/api/admin/users	Admin	All users list
GET	/api/ai/detect-anomalies	JWT	Anomaly detection
GET	/api/ai/predict-next-30-days	JWT	30-day forecast
GET	/api/ai/recommendations	JWT	Smart suggestions
GET	/api/ai/query	JWT	Groq LLM chat
GET	/api/health	No	Health check
WS	/ws/live	No	Real-time metrics

7. AI & ML FEATURES

a) Anomaly Detection — Z-Score Method

Flags abnormal spikes or drops in water/electricity usage. A Z-score measures how many standard deviations a value is from the mean.

```
z = (value "  mean) / stdDev

If |z| > 2.5  !’  flagged as ANOMALY
  value > mean  !’  labeled SPIKE
  value < mean  !’  labeled DROP

A Z-score > 2.5 covers ~98.8% of the normal distribution,
meaning only truly unusual values are flagged.
```

b) 30-Day Forecasting — Exponential Smoothing

Generates future predictions with confidence intervals. Gives more weight to recent data, adapts to trends smoothly.

```
Formula:  S_t = ;  r …÷B ² ƒ  !" ±) × S_(t-1)

;  Ò  ã2  ‡6Öö÷F†–ær f 7F÷"•
  !’ 30% weight on new observation
  !’ 70% weight on previous smoothed value

Seasonal adjustment: 7-day (day-of-week) cycle applied
Confidence interval: ±10% of predicted level
Output fields: predicted, lower, upper, confidence: 0.85
```

c) Groq AI — LLaMA 3-8B Language Model

- Model: llama3-8b-8192 via Groq API (fastest inference available)
- Temperature: 0.2 — near-deterministic, structured output
- System prompt forces JSON-only responses (no markdown)
- Used for natural language insights and recommendations
- Graceful fallback: if GROQ_API_KEY missing, feature disabled cleanly

d) Smart Recommendations Engine

- Rule-based engine comparing usage against averages and thresholds
- Analyzes historical patterns to generate actionable suggestions
- Example: "Reduce AC usage during peak hours to save ¹450/month"

Q: What is exponential smoothing?

A: A time-series method giving more weight to recent data. ; controls reactivity: high ; = reacts quickly, low ; = smoother. Used for 30-day demand forecasting.

Q: What is a Z-score?

A: Z = (x " ;Â• / <2à Measures standard deviations from the mean. |Z| > 2.5 means statistically unusual — occurs less than 1.2% of the time in a normal distribution.

8. REAL-TIME WEBSOCKET

```
// Server: broadcast live metrics every 10 seconds
const wss = new WebSocketServer({ server, path: "/ws/live" });

setInterval(() => {
  const payload = {
    type: "live_metrics",
    waterLpm:      18 + Math.random() * 6,   // litres/min
    electricityKw:  1.2 + Math.random() * 2.5, // kilowatts
    alerts: Math.random() > 0.9 ? 1 : 0,
    timestamp: new Date().toISOString()
  };
  wss.clients.forEach(client => client.send(JSON.stringify(payload)));
}, 10000);
```

Q: WebSocket vs HTTP?

A: HTTP is request-response — client must ask each time. WebSocket is full-duplex — server can push data anytime after a single handshake. Used here for live IoT metric streaming.

9. FRONTEND ARCHITECTURE

React 18 + Vite Setup

- React 18 functional components with hooks (useState, useEffect, useContext, useCallback)
- React Router v6 — client-side SPA routing, no page reloads
- Vite — ES module dev server with instant HMR, Rollup for production builds

3 Context Providers (wrap entire app)

Context	State Provided	Persisted
AuthContext	user, token, login(), register(), logout()	localStorage
ThemeContext	isDark, toggleTheme()	localStorage
LanguageContext	language, setLanguage()	localStorage

Protected Routing

DashboardLayout wraps all authenticated pages. On render it checks AuthContext for a valid token. If absent, redirects to /login using React Router's <Navigate> component.

Axios API Layer (services/api.js)

- Request interceptor: auto-attaches Bearer <token> to every outgoing request
- Response interceptor: catches 401 !' clears localStorage !' redirects to /login
- Timeout: 10 seconds per request
- Base URL: configurable via VITE_API_BASE_URL environment variable

Q: What is Vite?

A: A modern build tool using native ES modules for dev (instant HMR) and Rollup for production. 10-100x faster cold starts compared to webpack-based Create React App.

Q: What is useContext?

A: A React hook that consumes a Context value without prop drilling. Any component calls useContext(AuthContext) to get auth state from anywhere in the tree.

Q: useEffect vs useCallback?

A: useEffect runs side effects after render (data fetching, subscriptions). useCallback memoizes a function reference to prevent unnecessary re-renders when passed as prop.

10. SECURITY MEASURES (OWASP)

Threat	Mitigation Applied
SQL Injection	Parameterized queries ($1, $2 placeholders) — no string concatenation
Password Theft	bcrypt with salt (cost factor 10) — hashes are irreversible
CSRF	JWT in Authorization header (not cookies) — cannot be sent cross-site
CORS	Whitelist of allowed origins, rejects unknown domains
Unauthorized Access	JWT protect middleware on all private routes
Privilege Escalation	adminOnly middleware checks role === "admin" at server level
Secret Leakage	JWT_SECRET and DB credentials in .env, never committed to git
XSS	React auto-escapes JSX output, all responses are Content-Type: JSON
Request Flooding	10MB body limit on express.json, timeout on Axios (10s)
Info Leakage	Stack traces only sent in NODE_ENV=development

11. INDIA TARIFF CALCULATION

HydroGrid implements India-specific slab-based tariff for both electricity and water across multiple states. Each state has different rate slabs; cost is calculated progressively.

```
// Maharashtra Electricity Slabs
upto 100 units  !’  ¹4.41 / unit
upto 300 units  !’  ¹8.82 / unit
upto 500 units  !’  ¹11.72 / unit
above 500 units !’  ¹12.92 / unit

// Delhi Water: FREE upto 20,000 litres (domestic)
20,000 – 30,000 L !’  ¹0.03 / litre
above 30,000 L   !’  ¹0.05 / litre
```

States supported: Maharashtra, Delhi, Karnataka, Uttar Pradesh, Gujarat + default nationwide rates.

12. DEPLOYMENT

Docker / docker-compose

- 3 services: hydrogrid-api (Node), hydrogrid-web (Nginx + React build), mongodb
- Health checks on API service: GET /api/health every 30 seconds
- Services restart unless-stopped for auto-recovery
- Environment variables injected at runtime (not baked into image)

Cloud Options

Platform	File	Service
Render.com	render.yaml	Backend API + static frontend
Vercel	client/vercel.json	Frontend SPA deployment
Docker Hub	server/Dockerfile + client/Dockerfile	Containerized full-stack

13. COMMON VIVA QUESTIONS & ANSWERS

Q: What is CORS?

A: Cross-Origin Resource Sharing. Browser blocks requests from localhost:5173 to localhost:5000 by default. The server explicitly allows it using the cors middleware with a whitelist of origins.

Q: What is REST?

A: Representational State Transfer — uses HTTP verbs (GET/POST/PUT/DELETE), stateless requests, and resource-based URLs. Our API follows REST conventions throughout.

Q: Difference between SQL and NoSQL?

A: SQL is structured, relational, ACID-compliant (PostgreSQL). NoSQL is schema-less, document or key-value based (MongoDB). We use PostgreSQL for relational integrity.

Q: What is middleware in Express?

A: A function (req, res, next) => {} that runs between request and response. Examples: protect (JWT check), errorHandler, cors (headers), express.json (body parsing).

Q: What does bcrypt.genSalt(10) do?

A: Generates a cryptographic salt with cost factor 10 (2^10 = 1024 hashing rounds). Makes brute-force attacks computationally expensive. Salt is embedded in the hash string itself.

Q: What is Tailwind CSS?

A: Utility-first CSS framework — styles composed inline with classes like flex, p-4, text-blue-500. Zero unused CSS in production via tree-shaking (PurgeCSS).

Q: Difference between PUT and PATCH?

A: PUT replaces the entire resource. PATCH partially updates it. We use PUT /api/auth/profile for profile updates.

Q: How does role-based access work?

A: User's role stored in DB ('user' or 'admin'). JWT decoded !' user fetched !' req.user.role checked in adminOnly middleware. Returns 403 if role doesn't match.

Q: What is the difference between == and ===?

A: == allows type coercion ("1" == 1 is true). === checks both value and type ("1" === 1 is false). Always use === in JavaScript.

Q: What are React hooks?

A: Functions that let you use React state and lifecycle features in functional components. Key hooks used: useState, useEffect, useContext, useCallback.

Q: What is prop drilling and how did you avoid it?

A: Passing props through many component layers unnecessarily. Avoided using React Context API (AuthContext, ThemeContext, LanguageContext) for global state.

Q: What is a JWT token?

A: JSON Web Token — base64-encoded header.payload.signature. Server signs payload with JWT_SECRET. Clients send it in Authorization header. Stateless — no DB lookup needed to validate signature.

Q: What happens when a JWT expires?

A: jwt.verify() throws TokenExpiredError. The errorHandler returns 401. Frontend response interceptor catches it, clears localStorage, and redirects to /login.

Q: What is an Axios interceptor?

A: A function that intercepts every request or response. Request interceptor adds auth token. Response interceptor handles 401 errors globally without repeating code in every component.

14. WHAT MAKES YOUR PROJECT UNIQUE

- India-specific state-wise tariff slabs — real, accurate utility bills
- No heavy ML libraries — custom Z-score & exponential smoothing in pure JavaScript
- Dual AI system — local statistical engine + Groq LLaMA 3 for natural language
- WebSocket live metrics feed simulating real IoT sensor data
- Full gamification — badges, leaderboard, carbon footprint scoring
- Production-ready — Docker, Nginx, health checks, error handling, CORS, env management
- India GeoJSON state map for geographic consumption visualization
- Google OAuth alongside traditional JWT authentication

15. POSSIBLE IMPROVEMENTS (If Asked)

- Use httpOnly cookies for JWT instead of localStorage (prevents XSS token theft)
- Add rate limiting on auth endpoints using express-rate-limit
- Redis caching for leaderboard and dashboard aggregation queries
- WebSocket authentication — currently unauthenticated connections are accepted
- React Query for server state management (caching, background refetch)
- Unit and integration tests — Jest + React Testing Library
- HTTPS enforcement in production with SSL certificates via Let's Encrypt